

Scan to enter site
www.ktunotes.in



Get it on play store
bit.ly/ktunotesapp



mail@ktunotes.in

The logo for KTUnotes, featuring a stylized yellow and orange 'C' shape above the text.
KTUnotes
The learning companion.

Module 6

Distributed mutual exclusion – central server algorithm – ring based algorithm- Maekawa's voting algorithm – Election: Ring -based election algorithm – Bully algorithm

6.1 DISTRIBUTED MUTUAL EXCLUSION

Distributed processes often need to coordinate their activities. If a collection of processes share a resource or collection of resources, then often mutual exclusion is required to prevent interference and ensure consistency when accessing the resources. The application-level protocol for executing a critical section is as follows:

- **enter()**: enter critical section – block if necessary
- **resourceAccesses()** : access shared resources in critical section
- **exit()**: leave critical section – other processes may now enter

Our essential requirements for mutual exclusion are as follows:

- **ME1: (safety)** At most one process may execute in the critical section (CS) at a time.
- **ME2: (liveness)** Requests to enter and exit the critical section eventually succeed.

ME2 implies freedom from both deadlock and starvation. A deadlock would involve two or more of the processes becoming stuck indefinitely while attempting to enter or exit the critical section, by virtue of their mutual interdependence. But even without a deadlock, a poor algorithm might lead to starvation: the indefinite postponement of entry for a process that has requested it.

The absence of starvation is a fairness condition.

- **ME3: (ordering)** If one request to enter the CS happened-before another, then entry to the CS is granted in that order.

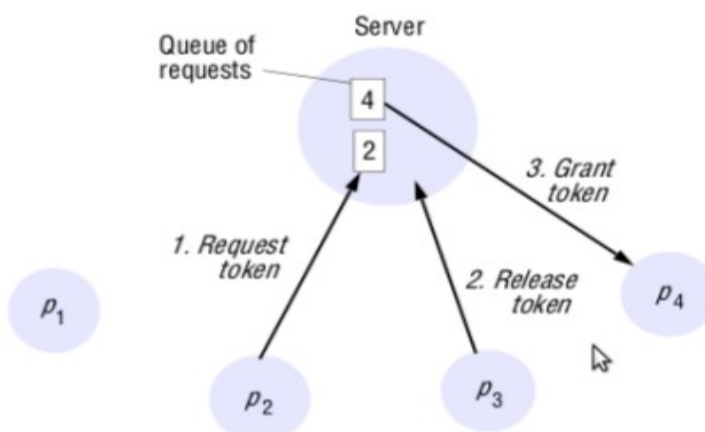
We evaluate the performance of algorithms for mutual exclusion according to the following criteria:

- the **bandwidth** consumed,
- the **client delay** incurred by a process at each entry and exit operation;
- the algorithm's effect upon the **throughput** of the system

Algorithms for mutual exclusion

- A central server algorithm
- A ring-based algorithm
- An algorithm using multicast and logic clocks (Ricart and Agarwal)
- Maekawa's voting algorithm

A central server algorithm



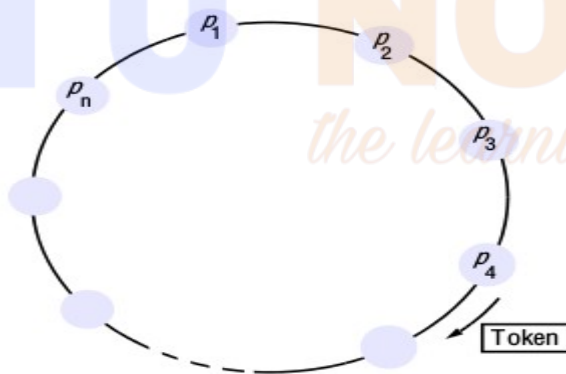
The simplest way to achieve mutual exclusion is to employ a server that grants permission to enter the critical section. To enter a critical section, a process sends a request message to the server and awaits a reply from it. Conceptually, the reply constitutes a token signifying permission to enter the critical section. If no other process has the token at the time of the request, then the server replies immediately, granting the token. If the token is currently held by another process, then the server does not reply, but queues the request. When a process exits the critical section, it sends a message to the server, giving it back the token.

Four processes p_1, p_2, p_3, p_4

- p_1 - no need to enter critical section (CS)
- p_4 & p_2 send request the token to enter CS
- But p_3 is accessing CS. So p_4 & p_2 enter the queue.
- p_3 released its token
- p_4 get the permission to enter CS
- After p_4 released, permission grant to p_2

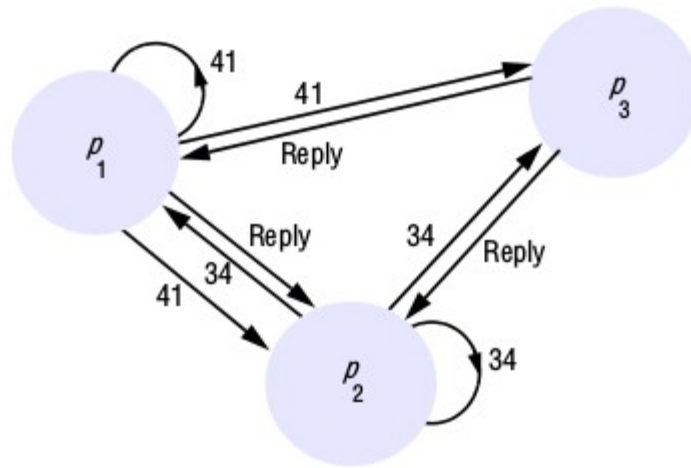
A ring-based algorithm

One of the simplest ways to arrange mutual exclusion between the N processes without requiring an additional process is to arrange them in a logical ring. This requires only that each process p_i has a communication channel to the next process in the ring, $p_{(i+1) \bmod N}$. The idea is that exclusion is conferred by obtaining a token in the form of a message passed from process to process in a single direction- clockwise. If a process does not require to enter the critical section when it receives the token, then it immediately forwards the token to its neighbour. A process that requires the token waits until it receives it, but retains it. To exit the critical section, the process sends the token on to its neighbour.



An algorithm using multicast and logic clocks (Ricart and Agarwal)

Ricart and Agrawala [1981] developed an algorithm to implement mutual exclusion between N peer processes that is based upon multicast. The basic idea is that processes that require entry to a critical section multicast a request message, and can enter it only when all the other processes have replied to this message. The conditions under which a process replies to a request are designed to ensure that conditions ME1–ME3 are met.



To illustrate the algorithm, consider a situation involving three processes, p_1 , p_2 and p_3 .

- p_3 is not interested in entering the critical section
- p_1 and p_2 request entry concurrently. Timestamp of p_1 's request is 41, and that of p_2 is 34
- p_3 receives their requests, it replies immediately
- p_2 receives p_1 's request, it finds that its own request has the lower timestamp and so does not reply
- p_1 finds that p_2 's request has a lower timestamp than that of its own request and so replies immediately.
- On receiving this second reply, p_2 can enter the critical section. When p_2 exits the critical section, it will reply to p_1 's request and so grant it entry.

Ricart and Agrawala's algorithm

On initialization

state := RELEASED;

To enter the section

state := WANTED;

Multicast request to all processes;

T := request's timestamp;

Wait until (number of replies received = (N - 1));

state := HELD;

On receipt of a request $\langle T_i, p_i \rangle$ at p_j ($i \neq j$)

if (state = HELD or (state = WANTED and $(T, p_j) < (T_i, p_i)$))

then

queue request from p_i without replying;

else

reply immediately to p_i ;

end if

To exit the critical section

state := RELEASED;

reply to any queued requests;

Maekawa's voting algorithm

Maekawa observed that in order for a process to enter a critical section, it is not necessary for all of its peers to grant it access. Processes need only obtain permission to enter from subsets of their peers. A subset contain $\{ p_1, p_2, p_3, p_4 \}$, if p_1 wants to enter CS it need only permission from p_2, p_3 and p_4 . ie. A process p_i send request msg to all other $k-1$ member of v_i .

This algorithm can tolerate some process crash failure. If a crashed process is not in a voting set then its

failure will not affect the other process.

On initialization

state := RELEASED;

voted := FALSE;

For p_i to enter the critical section

state := WANTED;

Multicast request to all processes in V_i ;

Wait until (number of replies received = K);

state := HELD;

On receipt of a request from p_i at p_j

if (state = HELD or voted = TRUE)

then

queue request from p_i without replying;

else

send reply to p_i ;

voted := TRUE;

end if

For p_i to exit the critical section

state := RELEASED;

Multicast release to all processes in V_i

On receipt of a release from p_i at p_j

if (queue of requests is non-empty)

then

remove head of queue – from p_k , say

send reply to p_k ;

voted := TRUE;

else

voted := FALSE;

end if

ELECTIONS

An algorithm for choosing a unique process to play a particular role (coordinator) is called an election algorithm. An election algorithm is needed for this choice. It is essential that all the processes agree on the choice. Afterwards, if the process that plays the role of server wishes to retire then another election is required to choose a replacement. We say that a process calls the election if it takes an action that initiates a particular run of the election algorithm. At any point in time, a process p_i is either a participant – meaning that it is engaged in some run of the election algorithm – or a non-participant – meaning that it is not currently engaged in any election.

Each process p_i ($i = 1 \# 2 \# \dots \# N$) has a variable elected_i , which will contain the identifier of the elected process. When the process first becomes a participant in an election it sets this variable to the special value 'A' to denote that it is not yet defined.

Our requirements are that, during any particular run of the algorithm:

E1: (safety) A participant process p_i has $\text{elected}_i = A$ or $\text{elected}_i = P$, where P is chosen as the non-crashed process at the end of the run with the largest identifier.

E2: (liveness) All processes p_i participate and eventually either set $\text{elected}_i = A$ – or crash.

Two algorithms,

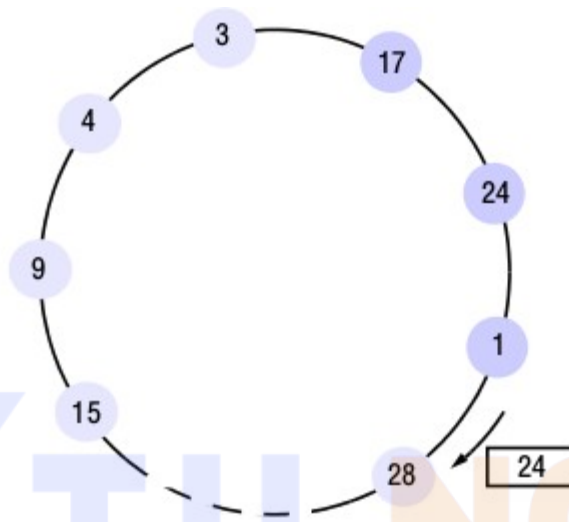
- **A ring-based election algorithm**
- **Bully algorithm**

A ring-based election algorithm

The algorithm of Chang and Roberts is suitable for a collection of processes arranged in a logical ring. Each process p_i has a communication channel to the next process in the ring, $p_{(i+1)}$

mod N , and all messages are sent clockwise around the ring. The goal of this algorithm is to elect a single process called the coordinator, which is the process with the largest identifier.

Initially, every process is marked as a non-participant in an election. Any process can begin an election. It proceeds by marking itself as a participant, placing its identifier in an election message and sending it to its clockwise neighbour. When a process receives an election message, it compares the identifier in the message with its own. If the arrived identifier is greater, then it forwards the message to its neighbour. If the arrived identifier is smaller and the receiver is not a participant, then it substitutes its own identifier in the message and forwards it; but it does not forward the message if it is already a participant. On forwarding an election message in any case, the process marks itself as a participant. If, however, the received identifier is that of the receiver itself, then this process's identifier must be the greatest, and it becomes the coordinator. The coordinator marks itself as a non-participant once more and sends an elected message to its neighbour, announcing its election and enclosing its identity.



- The election was started by process 17. Process forward to neighbour with greatest identifier.
- The election message currently contains 24, and forwards
- but process 28 will replace this with its identifier when the message reaches it.
- The received identifier is that of the receiver itself, then this process's identifier must be the greatest, and it becomes the coordinator and sends an elected message to its neighbour,

The bully algorithm

There are three types of message in this algorithm: an election message is sent to announce an election; an answer message is sent in response to an election message and a coordinator message is sent to announce the identity of the elected process.

The process that knows it has the highest identifier can elect itself as the coordinator simply by sending a coordinator message to all processes with lower identifiers. On the other hand, a process with a lower identifier can begin an election by sending an election message to those processes that have a higher identifier and awaiting answer messages in response. If none arrives within time T , the process considers itself the coordinator and sends a coordinator message to all processes with lower identifiers announcing this. Otherwise, the process waits a further period T_c for a coordinator

message to arrive from the new coordinator. If none arrives, it begins another election.

If a process p_i receives a coordinator message, it sets its variable $elect_i$ to the identifier of the coordinator contained within it and treats that process as the coordinator.

If a process receives an election message, it sends back an answer message and begins another election – unless it has begun one already.

When a process, P, notices that the coordinator is no longer responding to requests, it initiates an election.

- P sends an ELECTION message to all processes with higher no.
- If no one responds, P wins the election and becomes a coordinator.
- If one of the higher-ups answers, it takes over. P's job is done.

When a process gets an ELECTION message from one of its lower-numbered colleagues:

- Receiver sends an OK message back to the sender to indicate that he is alive and will take over.
- Receiver holds an election, unless it is already holding one.
- Eventually, all processes give up but one, and that one is the new coordinator.
- The new coordinator announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.

If a process that was previously down comes back:

- It holds an election.
- If it happens to be the highest process currently running, it will win the election and take over the coordinator's job.
- Biggest guy" always wins and hence the name "bully" algorithm.

Example: Bully Election

